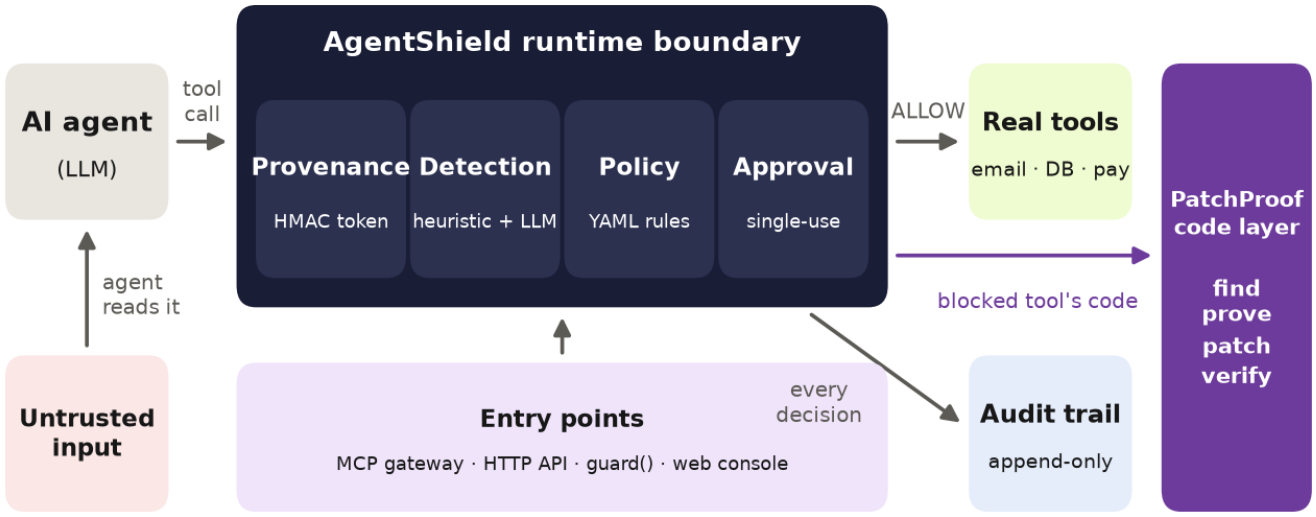
AgentShield: secure the agent, not just the prompt

AI agents read untrusted content (documents, emails, web pages) and then call real tools: send email, query customer data, move money. A single hidden instruction in a document can turn an obedient agent into an attacker. AgentShield sits at the tool-call boundary, below the model: every call is inspected before it runs and gets an explicit, audited decision of ALLOW, DENY or REQUIRE_APPROVAL, failing closed when unsure. When it blocks an attack on a tool, a second layer, PatchProof, scans that tool's own code, proves each weakness with a real exploit, patches it and verifies the fix. We assume an injection may succeed, and make sure it still can't do anything.

4.3% attacks succeed with enforcement ON (100% with it OFF, 84-case benchmark)	**0.0%** benign calls wrongly denied	**100%** high-impact calls held for a human	**0.54 ms** median decision time
8 → 0 exploitable vulnerabilities, before → after the closed loop	**85/85** exploit payload variants blocked after patching	**8/8** acceptance gates passed (independent grader)	**300 passed** automated tests

System architecture



Every entry point (MCP gateway, HTTP API, in-process guard and the web console) goes through the same engine. Provenance decides who really asked; detectors look for injected intent; the YAML policy composes rules with most-restrictive-wins; high-impact actions wait for a single-use human approval; every decision is written to an append-only audit trail.

Live demos in the console (all calls hit the real gateway)

Demo	What it shows
Homepage · Try a decision	One click sends a real tool call; the decision, its reasons and the audit call ID come back.
Document Lab	Upload a PDF (hidden white-text instructions, a reworded request, a Hindi UPI scam): every action the document triggers is decided, then the attacked tool's code is exploit-proven, patched and verified.
MCP Gateway	Compose any MCP tool call (tool, arguments, text, origin) and watch the decision change as you edit it; try to fake user trust without a signed token.
Overview · Evidence · Approvals	Live counts and charts from the audit trail, the benchmark, the closed loop, and single-use human approvals.

The problem and how we solved it

Prompt filters try to stop bad text from reaching the model, but the attacker and the defender write to the same prompt, so filters can be talked around. We moved the defence to the place the model cannot reason past: the moment it tries to act. Five mechanisms work together:

Layer	What it does	Why it matters
1 · Provenance	Every call carries its origin. Only a call signed by the agent harness with a single-use, expiring HMAC token counts as the user; everything else is UNTRUSTED. Replayed or forged tokens are rejected.	A document cannot claim to be the user. Verified live: claim without token → untrusted; replay → rejected.
2 · Detection	Keyword/structure heuristic (English + Hindi, Kannada, Tamil, Telugu lures, encoded and obfuscated payloads) plus an optional LLM detector (Sarvam-105b) for reworded attacks.	Cheap, deterministic first line; the LLM layer catches polite attacks with no trigger words.
3 · Policy	Per-tool YAML: sensitivity, allowed sources, approval requirement, disabled tools; defaults for unknown tools and detector errors. Rules compose; the most restrictive verdict wins.	Readable by a security reviewer, SHA-256 integrity-checked, fail-closed by default.
4 · Approval	High-impact actions (e.g. external email) pause for a human. Approvals are single-use and bound to a hash of the exact arguments.	An approved call cannot be replayed or edited afterwards.
5 · Audit	Append-only JSONL trail of every decision, its reasons and detector scores, with secret redaction.	Forensics, compliance evidence, and the data behind every chart in the console.

Problem-statement coverage (PS-01, runtime)

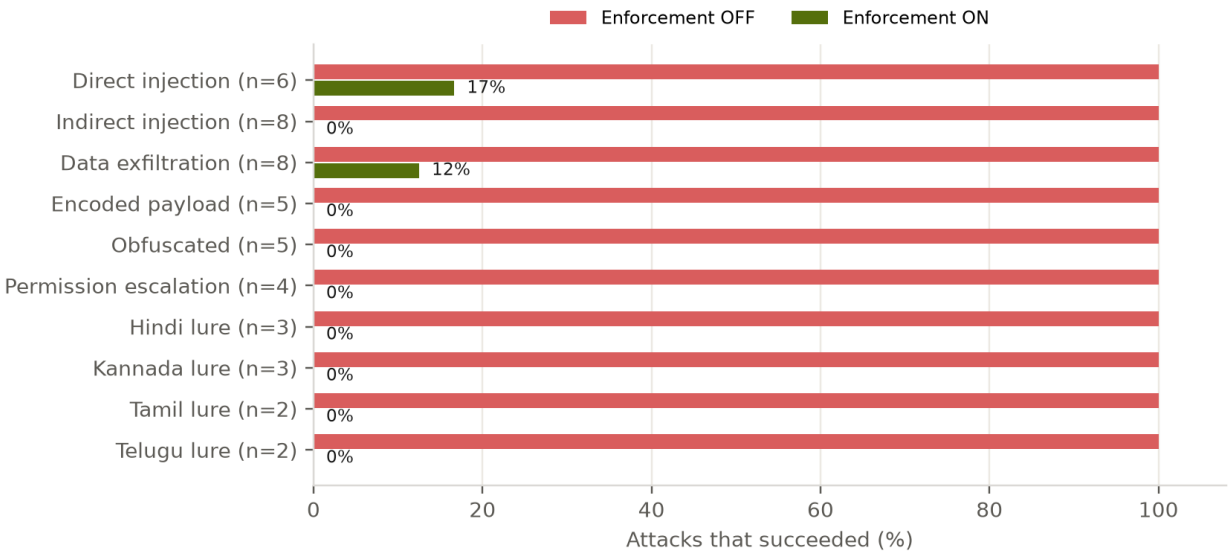
Requirement	How AgentShield meets it	Where to see it
Simulated agent, ≥2 tools, enforcement point, audit log	Mock agent + LangGraph agent; 6 governed tools; engine as PEP; JSONL audit	Overview, Audit & Forensics
Legitimate task	Benign calls allowed: 34/34 benign benchmark cases	Try a decision · search
Indirect prompt injection	8 cases, 0 succeed with enforcement ON	Document Lab (hidden white text)
Unauthorized sensitive-data request	read_customer_db only from user/system; untrusted → DENY	MCP Gateway composer
Tool invocation beyond permission	transfer_funds disabled; permission-escalation cases blocked	Document Lab · invoice
ALLOW / DENY / REQUIRE_APPROVAL + reasons	Every decision lists the rules that fired with detail text	Every page
User confirmation for high-impact actions	send_email always held; single-use, argument-bound approvals	Approval Center
Fail closed	Unknown tool, detector error or untrusted high-risk call → hold or deny	Policy Manager
Extensions: MCP gateway, signed provenance, audit trail, approval queue	MCP gateway + drop-in stdio proxy; HMAC tokens; trace IDs; approval queue	MCP Gateway page

PS-02 integration (code layer). The runtime block is also a signal: the tool that was attacked has a registered implementation file, so the bridge hands it to PatchProof, which finds the weakness, proves it with an exploit, applies a minimal patch and accepts it only if the exploit is now blocked and the regression suite stays green. The loop repeats until nothing exploitable remains.

Models and detection

Model / component	Role	Notes
Heuristic detector (rule-based)	Primary injection detector on every call	Override, persona, secrecy, exfiltration, encoded/obfuscated patterns; Hindi/Kannada/Tamil/Telugu lures; UPI payment-target exfil. Deterministic, ~0.5 ms.
Sarvam-105b (LLM, via OpenAI-compatible API)	Optional second detector for reworded attacks	Scores intent 0-1 with a short reason. Used by the Document Lab stack; measured 2-10 s per call, so the main gateway stays keyword-only. Providers also wired: Gemini, Groq, OpenRouter, NVIDIA NIM, custom.
Chroma default ONNX embedding	Local vector search over the untrusted RAG corpus	Runs on-device; retrieved text is labelled UNTRUSTED and policed at use time.
LangGraph StateGraph agent	Reference agent with human-in-the-loop interrupts	SQLite checkpointer; approval pauses the graph until a human decides.
PatchProof AST scanner + exploit harness	Code layer: find → prove → patch → verify	AST engine (Semgrep optional); exploits run only against bundled synthetic fixtures in an isolated process.
Core data models	ToolCall · EnforcementResult · PolicyReason · DetectionResult · SecurityEvent	Typed (Pydantic); one SecurityEvent stream joins runtime and code findings.

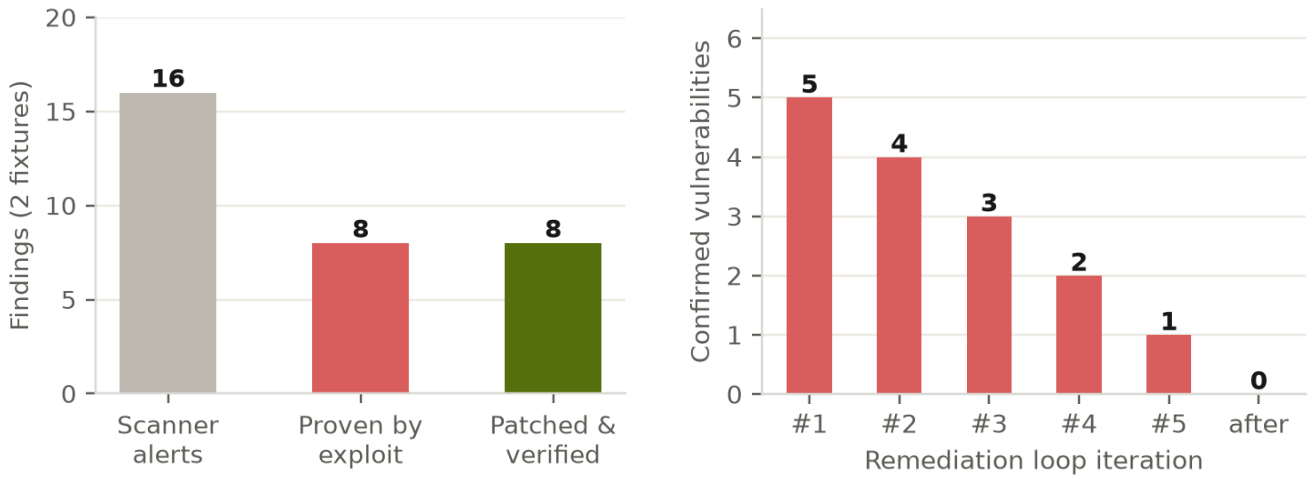
PS-01 benchmark: attacks that got through, by attack type



Frozen benchmark of 84 synthetic cases (46 malicious, 34 benign, 4 legitimate high-impact), run twice through the same engine: enforcement OFF lets every attack through; ON stops all but 2 of 46 (4.3%). With enforcement ON, 82.6% of attacks are denied and 13.0% are held for a human. The two that got through (D-06, E-07) are user-sourced requests to the low-risk search tool that try to extract the system instructions or an API key; the benchmark expected both to be held for approval. The keyword detector missed that phrasing and policy allows low-risk searches: this is the class the optional LLM detector layer targets.

PS-02 code layer: prove, patch, verify

Scanners are noisy. PatchProof only reports a weakness after an executable exploit confirms it, then accepts a patch only if the same exploit, plus every payload variant, is blocked and the regression suite stays green. Unproven alerts are dropped instead of being handed to a developer.



Left: across two fixtures, 16 scanner alerts became 8 proven weaknesses, all 8 patched and verified (precision 50% → 100%; 85/85 payload variants blocked). Right: the closed loop on the agent's own tool code, one confirmed vulnerability removed per iteration, converging in 5 iterations with 0 residual.

Proven and fixed weaknesses (VulnBank fixture)

CWE	Weakness	Severity	Location	Exploit confirmed	Patch accepted
CWE-89	SQL injection	critical	patchproof/vulnbank/app.py:111	yes	yes
CWE-639	Insecure direct object reference	high	patchproof/vulnbank/app.py:119	yes	yes
CWE-78	OS command injection	critical	patchproof/vulnbank/app.py:143	yes	yes
CWE-22	Path traversal	high	patchproof/vulnbank/app.py:150	yes	yes
CWE-79	Cross-site scripting	medium	patchproof/vulnbank/app.py:157	yes	yes

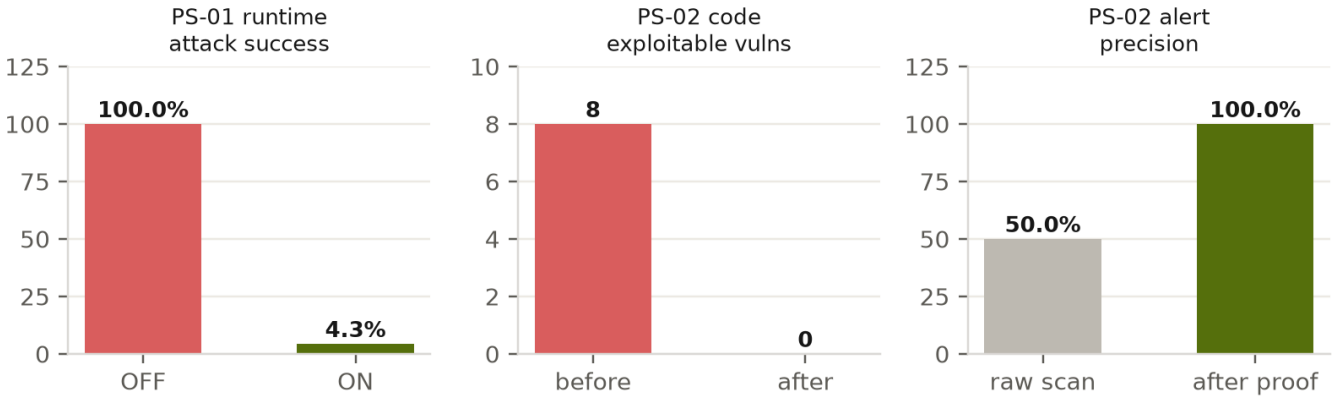
Independent grading (reports/scorecard.json)

Acceptance gate	Result	Target
Malicious calls stopped	PASS · 5/5	100% (DENY or REQUIRE-APPROVAL)
Benign false positive rate	PASS · 0.0	<= 5%
High impact approval correctness	PASS · 1/1	100% held for approval
Planted CWE scanner recall	PASS · 5/5	5/5
Confirmed CWEs fixed and verified	PASS · 5/5 verified	5/5 verified
Regressions green	PASS · 70/70 checks	all regression checks pass
Loop converges with zero residual	PASS · status=converged, residual=0, iterations=5	converged, residual=0, <= 8 iterations
Inspectable runtime decisions	PASS · 9/9 decisions reasoned	every decision has a reason

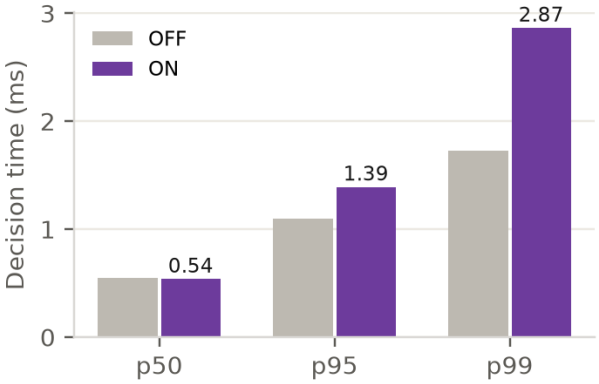
Mean verification time 1.6 s per patch; full code pipeline 17.4 s; mean coverage of patched functions 92.9%.

PS-01 vs PS-02: what each benchmark measures

The two layers defend the same asset (the agent's tools) from opposite sides, so they are measured differently: PS-01 measures how many attacks get through the boundary at runtime; PS-02 measures how many exploitable weaknesses remain in the tool code. Each panel below has its own axis.



	PS-01 · runtime boundary (AgentShield)	PS-02 · code layer (PatchProof)
Question answered	Does a malicious tool call get executed?	Does an exploitable weakness remain in the tool's code?
Dataset	84 frozen synthetic tool calls (46 malicious incl. 10 regional-language lures)	2 vulnerable fixtures (VulnBank, ShopAPI), 16 scanner alerts, 85 exploit variants
Method	Same engine run with enforcement OFF and ON; compare verdict to expected label	Scan → executable exploit → minimal patch → re-exploit + regression; loop until residual = 0
Headline before → after	Attack success 100% → 4.3%	Exploitable weaknesses 8 → 0; precision 50% → 100%
False positives	0.0% of benign calls denied or held	0 unproven alerts reported (dropped by the exploit gate)
Accuracy vs expectation	96.4% of verdicts match the expected label	Scanner recall 100% · exploit confirmation 100% · patch acceptance 100%
Speed	0.54 ms median, 2.87 ms p99 per decision	1.6 s to verify one patch; 17.4 s full pipeline
What it can't do alone	Cannot fix the underlying bug; only stops the call	Cannot stop a live attack in progress



Overhead is small. The median decision time is unchanged (0.55 ms OFF vs 0.54 ms ON). The slowest 1% rises from 1.73 to 2.87 ms, about one extra millisecond, which is negligible next to the LLM call that produced the tool call. Together the layers close the loop: runtime blocks generate the signal, code fixes remove the cause, and the re-attack proves it stays fixed.

APIs, technology and our assessment

Endpoint	Purpose
POST /v1/check	Decide one tool call (allow / deny / approval + reasons)
POST /v1/mcp/call · /demo	Real MCP session through the enforcement middleware
POST /v1/trust/demo	No token vs signed vs replayed token
POST /v1/document/analyze	Upload → runtime decisions → code fix
GET/POST /v1/approvals…	Pending queue; single-use decisions (admin)
POST /v1/redteam/run	Isolated OFF vs ON attack run (admin)
GET /v1/metrics · threats · agents	Live console analytics from the audit trail
GET /v1/bench/latest	Benchmark results
GET /v1/evidence/scorecard	Closed-loop gates and iterations
GET /v1/policy · tools · audit	Policy, inventory, append-only trail
MCP stdio server	python -m agentshield.gateway.mcp_gateway

Layer	Technology
Gateway	Python 3.11, FastAPI, Pydantic, Uvicorn
Agent protocol	Model Context Protocol (official SDK), stdio proxy
Detection	Heuristic engine; Sarvam-105b via OpenAI-compatible API
Agent & RAG	LangGraph, ChromaDB (local ONNX embeddings)
Code layer	PatchProof AST scanner, Flask fixtures, SARIF output
Documents	pypdf (bounded, sandboxed extraction)
Console	Next.js 16, React 19, TypeScript; server-side proxy
Security	HMAC provenance, httpOnly admin session, CSRF same-origin, route allowlist
Delivery	Docker, hash-pinned lockfile, Railway/Render configs

Our assessment

	Assessment
Strengths	Enforcement sits where the model cannot argue with it; every claim on screen comes from live gateway data or recorded results; trust is cryptographic, not self-declared; the code layer only reports what it can exploit, then proves the fix; regional-language lures are covered and measured.
Honest limits	Two benchmark attacks (4.3%) still get through: prompt-leak and secret requests sent to a low-risk tool. The LLM layer caught our reworded test attack but costs 2-10 s per call, so it runs only where latency is acceptable. The token replay cache is per-process (multi-worker needs a shared store). The MCP proxy supports stdio servers, not HTTP/SSE yet. All data is synthetic; TLS and secret storage are operator responsibilities.
Next steps	Faster LLM detector (smaller or local model) to run on every high-sensitivity call; shared replay cache (Redis); HTTP/SSE MCP transport; policy editing with signed, audited publishing; real-tool pilots.
Verdict	A working, measured defence for the attack that matters most for agents (indirect injection driving real tools), with a second layer that turns each blocked attack into a verified code fix.

Reproduce the numbers

What	Command (from the repository root)
Test suite	.venv/bin/python -m pytest -q
Runtime benchmark (84 cases, OFF vs ON)	python scripts/benchmark.py
Code layer + closed loop + grading	python -m secureai eval --once
Live MCP session in a terminal	python -m examples.mcp_demo
Gateway + console	uvicorn agentshield.gateway.app:app --port 8000 · cd web && npm run build && npm start